ЛР 1. Как пост сработал относительно обычного результата автора

Ростислав Манзюк, GitHub: grinchaus
Social Media Analytics, Agents & LLMs, ИТМО

Что нужно было сделать

Здесь задача - определить, получил ли LinkedIn-пост меньше реакций, чем автор обычно получает, примерно столько же или заметно больше. Классы - "below", "typical" и "above". Сравниваем пост с историей конкретного автора, поэтому большая аудитория сама по себе не означает успешный пост.

Обычный уровень - медиана log(1 + reactions) по предыдущим собственным постам автора, которых должно быть хотя бы пять. Основная метрика - macro-F1: каждый из трёх классов получает одинаковый вес. Заодно смотрю QWK, accuracy и матрицу ошибок.

Данные и проверка качества

Взял официальный релиз Kairos: 71,395 постов от 3,295 авторов в train и 17,623 постов от 824 авторов в test. Внутри train отдельно оставил 659 авторов для валидации (14,548 постов, seed 42). Авторы между частями не пересекаются.

Сначала проверил воспроизводимость разметки штатным kairos.label. Параметры моделей и смешивания подбирал только на авторах из train. Словарь TF-IDF, заполнение пропусков и масштабирование тоже обучаются внутри train. После выбора параметров зафиксировал конфигурацию, переобучил модели на всём train и посчитал итоговые результаты на официальном test.

Дополнительные подходы сравнил на двух разбиениях внутри train, seed 42 и 137, по 20% авторов для валидации. Конфигурации смешивания с первой валидации переносил на вторую. Итог выбрал по среднему macro-F1 с условием улучшения обеих валидаций. Разбиения зависимы: часть авторов и постов в них совпадает. Это две проверки, а не два независимых теста.

В своих признаках исправил разбор времени сбора: смешанные ISO-форматы с долями секунды и без них разбираются в UTC. Это восстановило 8 185 дат в размеченной части. Исходный baseline запускается с собственной предобработкой без изменений.

Какие подходы попробовал

Для начала воспроизвёл baseline - градиентный бустинг с балансировкой классов. Он использует длину и тип поста, хештеги, ссылки, время публикации и историю автора, но не читает слова в тексте.

Дальше добавил CatBoost и расширил признаки. Посчитал средние и разброс реакций за последние 3, 5, 10 и 20 постов, отклонение недавних результатов от обычного уровня, частоту публикаций и изменения длины текста. Заодно добавил признаки оформления: вопросы, восклицания, абзацы, символы, упоминания и долю заглавных букв.

В новом раунде добавил семантическую историю: результат похожих более ранних постов того же автора относительно его обычного уровня. Проверил 1, 3, 5 и 10 ближайших постов и плавные веса по косинусному сходству. Учитываются только уже известные реакции прошлых постов; текущий пост в историю не входит.

Отдельно попробовал текстовую модель: TF-IDF по словам и парам слов, а также по символьным фрагментам длиной 3-5. К ним добавил числовые признаки и обучил логистическую регрессию с балансировкой классов. Так модель может использовать и содержание, и стиль поста.

Заодно использовал вторую группу источников - профили авторов и метаданные вложений. Добавил навыки, описание профиля, отрасль, страну и язык, число мест работы и образований. Для вложений взял количество и размеры изображений, длительность видео, число страниц и извлечённый текст документов. Сами картинки и видео модель не читает.

После этого проверил смешивание вероятностей CatBoost и текстовой модели. Вес моделей и поправки к классам выбрал по macro-F1 на валидации. По test ничего не подбирал. Валидационная оценка после такого выбора может быть оптимистичной, поэтому основной результат ниже - на отдельном test.

Отдельно проверил нейросетевое представление текста. Использовал multilingual E5-small как фиксированный локальный encoder. Тексты постов и профилей не передавались во внешний API. Перед кодированием добавлял префикс "query: ", как рекомендуют авторы модели. Тексты длиннее 512 токенов обрезаются.

PCA сжал векторы постов до 64 признаков, а профилей - до 16. PCA обучается только на обучающей части, после чего признаки добавляются в CatBoost. Заодно добавил сходство поста с профилем и предыдущими постами автора. Будущие тексты в эту историю не попадают.

Для текстовой истории дополнительно закодировал 20,595 начальных постов. Это те пять постов каждого автора, на которых формируется исходный уровень и для которых ещё нет целевой метки.

Версия encoder зафиксирована: 614241f622f53c4eeff9890bdc4f31cfecc418b3.

Дополнительно обучил небольшую нейросеть на полных E5-векторах текущего текста, профиле, отличии текста от недавней и всей предыдущей истории, их взаимодействиях и числовых признаках. Классификатор: слои 256 и 128 нейронов, dropout 0.3, AdamW. Масштабирование и обработка категорий обучаются только на fit-авторах.

Итоговое переобучение: 2 эпохи, усреднение инициализаций [42, 1042]. Доля нейросети в итоговых вероятностях - 0.3. После смешивания применяются одинаковые множители классов.

Что получилось

Подход	Val macro-F1	Test macro-F1	QWK	Accuracy
Baseline	0.5587	0.5403	0.4603	0.5445
CatBoost + E5	0.6445	0.6311	0.5835	0.6406
TF-IDF + признаки	0.6114	0.5931	0.5319	0.6105
Нейросеть на E5	0.6131	0.6072	0.5675	0.6108
Предыдущая версия	0.6662	0.6535	0.5917	0.6871
Смешивание	0.6766	0.6593	0.6018	0.6916

Валидация по авторам	Предыдущая версия	Выбранная модель
seed 42	0.6662	0.6766
seed 137	0.6564	0.6729

Относительно уже отправленной версии прирост macro-F1 - +0.0058, 95% парный интервал по авторам - [+0.0014; +0.0102]. Test ранее уже использовался для оценки первой версии. В новом раунде кандидаты выбирались без его оценивания, а итоговое сравнение выполнено после фиксации настроек.

Также проверил дообучение двух верхних слоёв E5 вместе с классификатором текста и контекста. Результаты этого более тяжёлого подхода сохранены с остальными экспериментами. Итоговая конфигурация выбрана по согласованному улучшению двух валидаций.

В README исходного репозитория указан baseline 0.5410. В таблице использую результат его отдельного запуска в этом окружении - 0.5403.

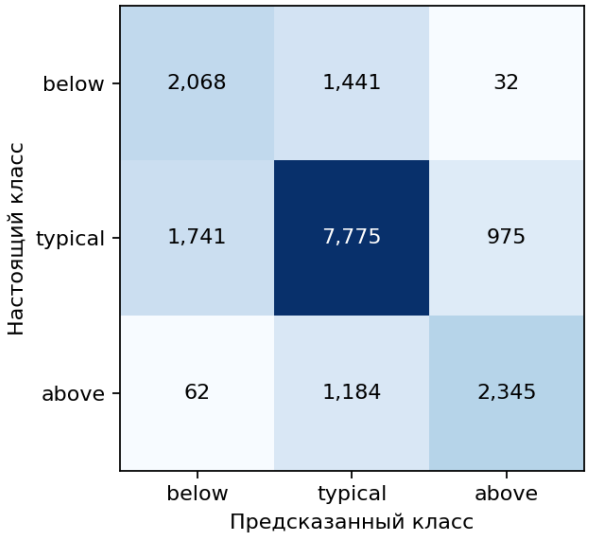
Итоговый macro-F1 - **0.6593**. Разница относительно воспроизведённого baseline - **+0.1190**. Обе модели оценены на одном и том же полном test.

95% доверительный интервал macro-F1 - [0.6498; 0.6685]. Интервал разницы с baseline - [+0.1095; +0.1284]. Посчитал парным bootstrap по авторам, 2000 повторов. Посты одного автора при этом остаются вместе.

В итоговой конфигурации вес CatBoost по классам - [0.9, 0.5, 0.9], множители вероятностей классов - [0.75, 1.0, 0.6], число итераций CatBoost - 2162.

Глубина дерева - 6, C логистической регрессии - 1.0.

Класс	Precision	Recall	F1	Постов
below	0.5342	0.5840	0.5580	3541
typical	0.7476	0.7411	0.7443	10491
above	0.6996	0.6530	0.6755	3591



Почему это может работать и где ограничения

История показывает, насколько стабилен автор и не изменился ли его обычный результат в последнее время. Текстовая модель добавляет информацию, которой baseline не пользуется. Сравнение моделей показывает полезность целых подходов, но не доказывает вклад каждого отдельного признака - для этого нужна более подробная абляция.

Реакции текущего поста, его residual, label и идентификаторы автора и поста не попадают в матрицу признаков. Окна истории сдвинуты на один пост назад. Проверки меняют текущие и будущие реакции и подтверждают, что признаки текущего и более ранних постов остаются прежними. Коды людей и компаний в тексте нормализованы.

Есть ограничение самого набора: реакции - это снимок на момент сбора данных, а число подписчиков измерено позже публикации. Возраст поста на момент сбора тоже используется как признак, хотя он связан с процедурой сбора данных. Поэтому результат относится к этому офлайн-заданию и не означает, что столько же получится при прогнозе сразу в момент публикации.

Профиль автора тоже является снимком на момент сбора. Его содержание может отличаться от того, что было в профиле при публикации старых постов. Это отдельное ограничение использования профиля как второго источника.

Историю test-автора использую так же, как официальный baseline: более ранние посты с известными реакциями могут участвовать в признаках более поздних. Картинки и видео в этой версии не использовал.

Как воспроизвести

Python 3.9.6, scikit-learn 1.6.1. Дополнительное обучение и оценка заняли 171 с. Кодирование текстов - отдельный подготовительный шаг. После установки requirements-embeddings.txt и распаковки данных достаточно запустить:

```
python run_lab1_v2.py --data data
```

Команда проверяет разметку, готовит локальные эмбеддинги, обучает модели с зафиксированными параметрами из selection.json, проверяет предсказания и собирает комплект для сдачи.

В репозитории лежат notebook, предсказания и метрики. Обученную модель можно сохранить локально и проверить повторный inference без переобучения. Данные и пароли в репозиторий не загружены.

Материалы: официальный baseline, датасет Kairos, слайды и уточнения по сдаче из чата курса.

Encoder: intfloat/multilingual-e5-small, описание модели и правила кодирования.